

Orientação ao aluno para o Projeto Integrador II

Escopo temático: DevOps, Ciência de Dados e Computação em Nuvem

Finalidade

Este guia define o que a equipe deverá construir, documentar e comprovar no Projeto Integrador II. O projeto deve integrar dados, aplicação e ambiente de execução em nuvem, usando práticas de DevOps para tornar a entrega reproduzível. A automação industrial é opcional: pode enriquecer o projeto, mas não é requisito para a nota básica.

Entrega obrigatória

PDF obrigatório	Plano de execução e evidências do projeto em docs/plano-projeto-integrador-ii.pdf dentro do repositório.
Fonte editável	Manter Markdown, DOCX ou equivalente no repositório e gerar o PDF a partir da versão final.
Escopo principal	Pipeline, dados, aplicação/serviço, armazenamento e implantação em nuvem.
Escopo opcional	Automação industrial, sensores, simulador de máquina ou integração com dados de processo.

Resultado esperado

Ao final, o avaliador deve conseguir entender o problema, executar ou acessar a solução, verificar a origem e o tratamento dos dados, observar o pipeline e consultar as evidências no repositório.

Tema orientador

Construir um serviço de monitoramento e análise de um processo ou ativo, com dados reais, públicos ou simulados. O serviço deve receber dados, armazená-los, produzir indicadores e disponibilizar uma aplicação ou API em ambiente de nuvem. O pipeline deve automatizar validações e a publicação da versão de demonstração.

Fluxo mínimo

Fonte de dados → ingestão → tratamento e armazenamento → análise/indicadores → API ou aplicação → implantação em nuvem → logs e feedback.

Exemplo de roteiro: sistema de gestão escolar

O roteiro abaixo é apenas um exemplo para ajudar a equipe a entender o nível de detalhamento esperado. Ele não é o único tema possível e não precisa ser copiado. Outro tema será aceito se demonstrar o mesmo encadeamento entre dados, análise, nuvem e entrega DevOps.

Contexto do exemplo

Uma escola deseja acompanhar frequência, desempenho e ocupação das turmas. Hoje os dados estão espalhados em planilhas, são consolidados manualmente e não permitem identificar rapidamente alunos com risco de reprovação ou salas com baixa ocupação. A equipe propõe um serviço que recebe dados escolares, calcula indicadores e disponibiliza um painel para a coordenação.

Passo	O que os alunos devem fazer no exemplo
1. Pergunta do usuário	A coordenação quer saber: qual é a taxa de frequência por turma? Quais alunos estão abaixo do limite definido? Como está a ocupação das salas?
2. Dados de entrada	Arquivos CSV ou registros simulados de alunos, turmas, presenças, notas e salas. Cada registro deve possuir identificador, data e origem.
3. Tratamento	Validar datas, notas dentro do intervalo, presenças sem duplicidade e relacionamentos entre aluno, turma e período. Registros inválidos devem ser rejeitados ou marcados.
4. Armazenamento	Criar tabelas ou coleções para alunos, turmas, presenças, avaliações e salas. Documentar chaves, relacionamentos e consultas usadas pelos indicadores.
5. Indicadores	Calcular, por exemplo, frequência média por turma, percentual de alunos abaixo de 75% de presença, média de notas por período e taxa de ocupação das salas.
6. Serviço	Disponibilizar uma API ou painel que permita filtrar por turma e período, consultar os indicadores e visualizar uma lista de alertas.
7. Nuvem	Publicar o serviço e o banco em ambiente de demonstração. Configurar variáveis sem expor senhas e registrar como iniciar, testar e desligar o ambiente.
8. DevOps	Manter o código em Git, executar testes a cada alteração, construir o artefato, publicar uma versão identificável e guardar logs da execução.

Exemplo de requisitos de aceite

- Ao importar um arquivo válido, os registros aparecem no armazenamento e ficam disponíveis para consulta.
- Um arquivo com nota inválida ou presença duplicada é rejeitado ou sinalizado, com registro do motivo.
- O painel permite selecionar turma e período e exibe pelo menos dois indicadores calculados a partir dos dados.
- O serviço pode ser executado seguindo o README e possui uma URL, endpoint ou instrução de acesso para demonstração.
- Uma alteração no código dispara o pipeline; testes aprovados geram uma versão identificável.
- A equipe consegue mostrar um log de erro e explicar como investigaria ou recuperaria o serviço.

Exemplo de organização do repositório

Local	Conteúdo no exemplo
README.md	Problema, arquitetura, execução local/nuvem, indicadores e roteiro de demonstração.
docs/plano-projeto-integrador-ii.pdf	Plano final obrigatório em PDF.
data/sample/	Arquivos sintéticos de alunos, turmas, presenças e notas.
src/	API, processamento dos dados e regras dos indicadores.
tests/	Testes de validação, cálculo e integração do fluxo principal.
infra/ ou docker-compose.yml	Configuração reproduzível do serviço e do banco.
.github/workflows/ ou equivalente	Pipeline de build, testes e publicação.
docs/decisoões/	Registros de escolhas sobre dados, nuvem, segurança e escopo.

Extensão industrial no exemplo

A equipe pode simular sensores de ocupação, consumo de energia ou temperatura das salas e exibir esses dados junto aos indicadores escolares. A extensão é opcional, deve ser segura e não substitui o fluxo acadêmico nem o pipeline.

1 O que a equipe precisa fazer

Etapa 1 - Definir o problema

- Escolher um processo ou ativo e identificar um usuário que tomará decisões com os dados.
- Descrever a pergunta que a solução responderá e o valor esperado.
- Definir o que ficará fora do escopo para manter o projeto executável no semestre.
- Priorizar um fluxo funcional principal e registrar extensões como opcionais.

Etapa 2 - Planejar dados e arquitetura

- Descrever a origem, formato, frequência, volume e qualidade dos dados.
- Definir o modelo de armazenamento e as transformações necessárias.
- Desenhar o fluxo entre fonte, serviço, banco, aplicação e nuvem.
- Registrar variáveis de ambiente, dependências, riscos e critérios de aceite.

Etapa 3 - Implementar e versionar

- Manter o código em Git com commits pequenos e mensagens compreensíveis.
- Separar configuração de código e não versionar credenciais.
- Implementar validação de entrada, tratamento de erros e testes do fluxo crítico.
- Documentar como instalar, executar e verificar a solução.

Etapa 4 - Automatizar a entrega

- Criar pipeline para build, testes e verificações de qualidade.
- Empacotar ou configurar a aplicação para execução reproduzível.
- Publicar em ambiente de homologação ou demonstração em nuvem.
- Identificar a versão implantada e descrever como retornar à versão anterior.

Etapa 5 - Analisar e demonstrar

- Apresentar indicadores, limitações e decisões tomadas.
- Demonstrar uma alteração passando pelas verificações.
- Mostrar logs, métricas ou registros de execução.
- Comparar resultados com critérios de aceite e propor melhorias.

2 Escopo funcional e técnico

O escopo principal deve conter

Tema	Requisito mínimo
Dados	Recebimento de dados reais, públicos ou simulados, com validação e identificação de origem.
Tratamento	Limpeza, transformação ou agregação suficiente para gerar informação útil.
Armazenamento	Banco ou serviço equivalente, com modelo e consultas documentados.
Indicadores	Pelo menos dois indicadores com fórmula, fonte e interpretação.
Serviço	API, aplicação web ou serviço equivalente acessível para demonstração.
Nuvem	Ambiente remoto ou plataforma compatível, com instruções e variáveis de configuração.
DevOps	Versionamento, pipeline, testes, artefato identificável e observabilidade básica.

Automação industrial como escopo opcional

A equipe pode adicionar sensores, CLP, MQTT, simulador de máquina, coleta de telemetria, alertas de limite ou integração com um processo produtivo. Para receber reconhecimento, a extensão deve estar integrada ao fluxo principal e apresentar evidências, sem substituir os requisitos de dados, nuvem e entrega.

Nível opcional	Exemplo	Evidência
Básico	Simulador de sensor ou arquivo periódico	Dados gerados e recebidos pelo serviço
Intermediário	MQTT, API de dispositivo ou gateway	Mensagens, tópicos e tratamento documentados
Avançado	Integração com equipamento ou processo real autorizado	Demonstração segura, limites e autorização

Não conecte o sistema a equipamento crítico nem use dados industriais sigilosos sem autorização. Dados sintéticos ou anonimizados são aceitáveis e recomendados para a atividade acadêmica.

Entregáveis

Marco	Conteúdo
1. Proposta	Problema, usuário, valor, requisitos, limites e backlog.
2. Arquitetura	Diagrama, modelo de dados, fluxo de execução e critérios de aceite.
3. Incremento funcional	Dados processados, armazenamento, indicadores e serviço funcionando.
4. Entrega automatizada	Pipeline, testes, artefato/versionamento e implantação em nuvem.
5. Evidências finais	PDF, README, logs, resultados, demonstração e retrospectiva.

3 Evidências e verificações

O repositório deve permitir localizar cada evidência. Use README, links, caminhos de arquivos, número de issue, commit, pull request, execução do pipeline ou relatório.

Dimensão	Verificação principal	O que será observado
Problema e valor	Manual	Objetivo, usuário, indicador e resultado coerentes.
Qualidade dos dados	Manual + automática	Origem, validação, transformação, dados ausentes e registros de execução.
Código e testes	Automática	Build, testes, lint/análise e falhas bloqueantes.
Entrega e nuvem	Automática + demonstração	Configuração reproduzível, versão identificável, health check e acesso.
Segurança	Automática + manual	Secrets, dependências, entradas, permissões e dados protegidos.
Observabilidade	Automática + demonstração	Logs, métricas, alertas ou procedimento de diagnóstico.
Automação opcional	Manual	Integração segura, valor adicional e evidências da extensão.

O que pode ser verificado automaticamente

- Presença e abertura do PDF obrigatório.
- Build, testes unitários e testes do fluxo principal.
- Lint, análise estática, vulnerabilidades de dependências e detecção de secrets.
- Existência de pipeline e execução bem-sucedida.
- Construção de imagem/artefato, health check e endpoint de demonstração.
- Formato, esquema e regras básicas de qualidade dos dados.
- Rastreabilidade de versão, commits, tags e artefatos.

O que precisa ser verificado manualmente

- Se o problema é relevante e se os indicadores ajudam o usuário.
- Se as transformações dos dados fazem sentido e não escondem limitações.
- Se as decisões técnicas, riscos e custos foram justificados.
- Se a nuvem foi utilizada de forma adequada ao escopo.
- Se a apresentação demonstra domínio individual e não apenas reprodução de comandos.
- Se a automação industrial opcional é segura, coerente e integrada.

4 Critérios de avaliação

Critério	Peso	Verificação	Evidência
Problema, valor e requisitos	20%	Manual	Clareza do problema, usuário, valor, escopo e critérios de aceite.
Dados e indicadores	20%	Manual + automática	Origem, qualidade, tratamento, dois ou mais indicadores e interpretação.
Aplicação e nuvem	20%	Automática + manual	Serviço funcionando, armazenamento, configuração e implantação acessível.
DevOps e qualidade	25%	Automática	Git, pipeline, testes, análise, artefato, versão e recuperação.
Documentação e demonstração	10%	Manual	README, PDF, evidências, clareza e demonstração reproduzível.
Automação industrial opcional	5% bônus	Manual	Integração segura e valor adicional; não prejudica quem não escolher a extensão.

Matriz de composição da nota

Cada critério obrigatório recebe nível de 0 a 4. A pontuação é calculada por $(\text{nível} \div 4) \times \text{peso}$. A nota base soma 100 pontos. A automação industrial pode acrescentar até 5 pontos de bônus, limitada à nota máxima definida pela instituição.

Componente	Limite	Composição
Avaliação automática/repositório	45 pontos	Build, testes, segurança, pipeline, versão, dados e execução.
Avaliação manual	55 pontos	Problema, valor, decisões, indicadores, documentação e apresentação.
Bônus opcional	até 10 pontos	Automação industrial integrada, segura e demonstrada.

Exemplo: nível 3 em um critério de 20% gera $(3 \div 4) \times 20 = 15$ pontos. Se a equipe não implementar automação industrial, a nota base permanece 100% possível.

Regras de avaliação

- Sem o PDF obrigatório no repositório, a documentação não pode ultrapassar nível 1.
- Código sem teste ou pipeline executável perde a evidência automática correspondente.
- Evidência declarada, mas inexistente ou incompatível, não será pontuada como aplicada.
- A automação opcional só recebe bônus se não comprometer segurança, escopo principal ou reprodutibilidade.
- Cada estudante poderá explicar individualmente dados, nuvem, pipeline e decisões da equipe.

5 Checklist de entrega

	Antes de entregar, confirme
☐	docs/plano-projeto-integrador-ii.pdf existe, abre e está legível.
☐	README explica instalação, execução, arquitetura e links das evidências.
☐	Fonte editável do PDF está versionada e atualizada.
☐	Problema, usuário, escopo e critérios de aceite estão definidos.
☐	Dados possuem origem, formato, validação e tratamento documentados.
☐	Existem pelo menos dois indicadores com fórmula e interpretação.
☐	Aplicação ou API está acessível e conectada ao armazenamento.
☐	Pipeline executa build, testes e verificações compatíveis.
☐	Não há credenciais, tokens ou dados sigilosos no repositório.
☐	A versão implantada é identificável e há instrução de recuperação.
☐	Logs ou métricas permitem investigar uma falha.
☐	A equipe consegue demonstrar uma alteração passando pelo fluxo.
☐	Se adotada, a automação industrial está segura e documentada.
☐	A apresentação compara resultados com os critérios de aceite.

Orientação final

Construa primeiro um fluxo pequeno que funcione de ponta a ponta. Em seguida, automatize as verificações, publique em nuvem e só depois adicione extensões. A automação industrial é uma oportunidade de aprofundamento, não um substituto para o núcleo de dados, nuvem e DevOps.